Engineering Workbench — Report

Candidate: Arnav Mehta · Reg. no. RA2311056010137 · Effort: 32 hours · Handwritten appendix: `docs/handwritten/`

---

1. Design choices

One process, one engine. `Launch.exe` is a single statically linked C++17 program. It contains:

- the core library (`src/core`), which has no HTTP, JSON or UI dependency;
- a thin transport layer (`src/server`) on cpp-httplib and nlohmann/json.

The transport serves the HTML/JS interface to an Edge `--app` window on `127.0.0.1`. This satisfies "all backend logic in C++", keeps the UI responsive (requests and jobs run on worker threads) and needs no runtime installation on Windows 11.

I considered two alternatives:

- **Qt:** a large redistributable and licence obligations.
- **Electron:** ships a Node.js runtime and invites backend logic in JavaScript.

Separation of concerns.

- **Core:** core types are pure C++, and the core reports failures with typed exceptions (`ParseError` with a line number, `std::invalid_argument`, `DeviceTimeout`, `DeviceStateError`, `Cancelled`).
- **Transport:** a single `guarded()` wrapper maps those exceptions to HTTP 422/400/504/409 with the engine's message.
- **Frontend:** it validates inputs for immediate feedback, but every rule is enforced again in C++. The UI never computes results; it only draws them.
- **Exports:** CSV writers live in `core/report`, so the UI's export buttons and the command-line tool that produced `results/` emit identical files.

Long work. The benchmark is the only long job. `JobManager` runs it on a background thread and reports a stage and fraction complete. Cancellation is an atomic flag checked between cases. A job's result is published only on success, so cancelling cannot leave half-written results (walkthrough frames 11–12).

Interface. Six views share one layout:

- a sticky local navigation bar holding the view's primary actions;
- a short introduction;
- cards for inputs and results;
- a status bar with the last engine message, plus toasts for errors.

Charts are drawn by a small SVG module with labelled units, log axes and a hover read-out. The two 3-D views use three.js (orbit, pan, zoom, fit, preset views, picking).

---

2. Thermal FEA verification (Task C)

Problem. $-d/dx(kA\,dT/dx) = qA$ on $0 < x < L$, with:

- $L = 1$ m, $A = 10^{-4}$ m², $k = 10$ W/(m·K), $q = 10^{4}$ W/m³;
- $T(0) = 100$ °C, $T(L) = 20$ °C;
- steady state, constant properties, insulated lateral surface.

Discretisation. N uniform two-node linear elements, $h = L/N$.

- Element conductance: $K_e = (kA/h)[1\ -1;\ -1\ 1]$ (W/K).
- Consistent load: $f_e = (qAh/2)[1;\ 1]$ (W). This is exact for uniform q.

The global matrix is tridiagonal and is stored as three diagonals.

Boundary conditions. Both temperatures are prescribed. Nodes 0 and N are eliminated: their known contributions $K(i,0)\cdot T_0$ and $K(i,N)\cdot T_L$ move to the right-hand side of the neighbouring rows. This keeps the

reduced system symmetric positive definite, so it is solved by the Thomas algorithm in O(N). The boundary values are written back exactly, which a test asserts.

Hand check. For N = 2: $4 \cdot 10^{-3} \cdot T_1 - 2 \cdot 10^{-3} \cdot (100 + 20) = 0.5$ W, so $T_1 = 185$ °C = T_exact(0.5). This is a unit test.

Results (`results/fea_convergence.csv`; plots `fea_temperature.svg`, `fea_error_profile.svg`, `fea_convergence.svg`):

N	h [m]	max nodal error [°C]	max midpoint error [°C]	q h²/(8k) [°C]	observed order
10	0.1	5.7×10^{-14}	1.25	1.25	–
20	0.05	2.6×10^{-13}	0.3125	0.3125	2.000
40	0.025	7.7×10^{-13}	0.078125	0.078125	2.000
80	0.0125	1.1×10^{-12}	0.019531	0.019531	2.000

The midpoint errors are measured by evaluating the **FE interpolant** (linear shape functions) at every element midpoint and comparing with T_exact. The peak of T_exact is 188.2 °C at x = 0.42 m.

Debugging insight — why nodes alone would mislead. The nodal error is ~10^{-13} °C for every N (table above). A convergence check written against nodal values would therefore show flat round-off noise and could wrongly suggest either a perfect solver or a broken error measure.

The reason is that, in 1-D, linear elements with a consistent load reproduce the exact solution **at the nodes**: the Green's function of $-d^2/dx^2$ is piecewise linear, so it lies in the FE space. Between the nodes, the FE field is linear while the exact field is quadratic with curvature $-q/k$. The interpolation error of a quadratic peaks at the element midpoint at $(q/k)(h/2)^2/2 = q\,h^2/(8k)$, so halving h quarters the error (order 2). The measured midpoint errors match this formula to 10^{-14}.

Two consequences:

- the error metric must sample inside elements, as the brief requires;
- a nodal-only test cannot detect an assembly error that is also nodally exact.

That is why the suite also has the hand-calculated 2-element case, a q = 0 (exactly linear) case and an N = 200,000 round-off case.

Units and assumptions. All inputs are SI. Temperatures are in °C, and only differences enter K, so °C and K give the same field. The UI rejects non-positive L, A, k, N < 1 and temperatures below −273.15 °C, and the engine rejects them again. q may be negative (a heat sink) or zero.

Interdisciplinary adoption. Thermal finite-element analysis was outside my coursework, so I approached it the way I would approach any unfamiliar method: start from the physics, derive before coding, and make the analytical solution the test oracle. Integrating the ODE twice gave T_exact. Writing the weak form and integrating the products of linear shape functions gave K_e and f_e. A 2-element mesh solved by hand (T_1 = 185 °C) confirmed the assembly before any code ran. The surprising result, nodal values exact to round-off on every mesh, taught me that a numerical method must be checked where it is weakest (inside elements), not only where it is convenient. That same "independent check" habit carried over to the gradient check in Task F.

The verification route used in the code is:

1. derive the exact solution by integrating twice;
2. derive K_e and f_e from the weak form by integrating products of the linear shape functions;
3. check a 2-element system by hand;
4. use the analytical solution as the test oracle, measuring the error where the method is weakest (inside elements).

The same approach underlies the backpropagation check in §5.

---

3. Multicore performance (Task D)

Workload. `generateBatch(seed = 20260101, cases = 500, N = 10,000)` draws k ∈ [1, 400] W/(m·K) and q ∈ [10^3, 10^6] W/m³ from `std::mt19937_64`. Raw draws are mapped to [0, 1) by an explicit 53-bit conversion,

because `std::uniform_real_distribution` differs between standard libraries. A unit test pins the first draw, so cases are identical across compilers.

Execution paths. Serial and parallel paths call the same `fea::solve`.

- **Parallel:** a `WorkerPool` with a fixed number of `std::thread` workers and a bounded queue (capacity 2p; `submit` blocks when full). Tasks are chunks of 4 cases, and each case writes only its own pre-allocated slot. The output order is therefore the input order regardless of scheduling, with no locks on results.
- **Timed region:** solving and storing results (`std::chrono::steady_clock`). Case generation, file I/O, JSON, rendering and pool construction are excluded.
- **Repetitions:** each configuration does 1 warm-up and 5 measured runs; the median is reported.
- **Metrics:** T_1 is the one-worker pool run; speedup = T_1/T_p; efficiency = speedup/p.
- **Agreement:** each parallel configuration's final run is compared node by node with the serial result.

Machine (`results/bench_system.txt`):

Item	Value
CPU	Intel Xeon @ 2.80 GHz, 4 physical / 4 logical (cloud VM)
RAM	15.7 GiB
OS	Linux 6.18
Compiler	GCC 13.3.0, Release `-O3 -DNDEBUG`
Launch-to-ready	server listening ≈ 2–4 ms after process start; UI connected 1.8 s after launch (headless browser)

The Windows build under Wine connected after 7.2 s, most of which is Wine start-up.

Results (median s; four independent invocations, raw runs in `results/bench_runs.csv` and `results/bench_repeat/trial_*`):

p	Trial A	Trial B	Trial C	Trial D	Speedup range	Efficiency range
serial loop	0.228	0.295	0.235	0.211	–	–
1	0.195	0.295	0.214	0.192	1.00	100 %
2	0.185	0.203	0.157	0.159	1.05–1.45	53–73 %
3	0.130	0.126	0.127	0.143	1.34–2.34	45–78 %
4	0.114	0.109	0.101	0.117	1.64–2.71	41–68 %

The maximum $|T_{parallel} - T_{serial}|$ is **0 °C** in every configuration, well within the 10^{-8} °C requirement. The values are bit-identical because each case is solved by the same deterministic sequence on one thread; only the case-to-thread assignment varies.

Analysis. Parallel execution clearly helps (4 workers take about half the time of 1), but scaling is far from linear and noisy.

- **Memory-bound kernel.** Each solve allocates ~10 arrays of N doubles (≈ 0.9 MB) and streams through them a few times with O(1) arithmetic per element. Throughput is limited by memory bandwidth and the allocator; adding cores adds compute but not bandwidth.
- **Shared, virtualised CPUs.** The four vCPUs are not dedicated, so other tenants steal cycles. The same binary moved from 1.6× to 2.7× within minutes. That is why every repetition is recorded and medians are reported. The serial-loop median varies by 40 % between invocations for the same reason.
- **Overheads.** Pool start-up is excluded. Queue hand-off costs one mutex and condition-variable operation per chunk of 4 cases, which is negligible next to a ~0.4 ms solve.
- **A measurement bug found during development** (commit history). The first version created the pool inside the timed region, and the 1-worker pool was 50 % slower than the serial loop. Creating each pool once per worker count and reusing it for warm-up and measured runs removed that artefact.
- **Limitations and next steps:**
  - reusing per-thread solver buffers instead of allocating per case would cut memory traffic;
  - pinning threads, or running on a dedicated machine, would reduce variance.

No minimum speedup is claimed. The Performance view reports the reviewer's own machine, with exports.

4. Embedded integration (Task E)

`EMBEDDED_DESIGN.md` contains:

- the labelled block diagram;
- the interface and pin-voltage table: power, grounding, voltage domains, level shifting, connectors, host links, JTAG signal ownership;
- board-specific assumptions;
- a safe integration plan and limitations.

The design point the brief stresses: the JTAG probe is a **debug interface** whose levels follow the target's VREF. It is not an interchangeable compute board, and the software models devices by **capabilities** rather than assuming a common connector, voltage or protocol.

The C++ mocks implement discovery, capability reporting, connect/disconnect and a telemetry message for all three devices. The UI demonstrates two recoverable failures:

- a connect timeout recovered by automatic retry;
- a telemetry timeout recovered by reconnecting (frames 14–16).

Simulation is labelled on every device card, in the API payloads and in the logs.

5. Backpropagation (Task F)

Network: $z = wx + b$, $h = \tanh z$, $\hat{y} = vh + c$, $J = \tfrac{1}{2}(\hat{y} - y)^2$. Sample: $x = 1$, $y = 2$. Start: $w = 0.5$, $b = 0$, $v = 1$, $c = 0$. Learning rate $\eta = 0.1$.

Forward pass:

- $z = 0.5$
- $h = 0.462117$
- $\hat{y} = 0.462117$
- $J = 1.182542$

Backward pass:

- $\delta = \hat{y} - y = -1.537883$
- $\partial J/\partial v = \delta h = -0.710682$
- $\partial J/\partial c = \delta = -1.537883$
- $\partial J/\partial z = \delta \cdot v \cdot (1 - h^2) = -1.537883 \times 0.786448 = -1.209464 = \partial J/\partial w\ (x = 1) = \partial J/\partial b$

One simultaneous update, using the gradients from the original parameters:

- $w = 0.620946$
- $b = 0.120946$
- $v = 1.071068$
- $c = 0.153788$
- $J = 0.685774$

All four gradients are negative because $\hat{y} < y$, so descent increases every parameter and raises $\hat{y}$ toward y.

C++ check (`core/ml/backprop.cpp`, `results/backprop_gradients.csv`): the analytic gradients agree with central differences ($\varepsilon = 10^{-6}$) to $\le 1.2 \times 10^{-10}$, against a tolerance of 10^{-5}. Over 100 further steps the loss falls geometrically to 1.1×10^{-20} (`backprop_loss.svg`, a straight line on a log axis). Near the minimum the loss is locally quadratic, so gradient descent removes a roughly constant fraction of the error per step. One sample can be fitted exactly, so $J \to 0$; this demonstrates optimisation, not generalisation.

The handwritten derivation is in `docs/handwritten/`.

6. Tests and evidence

46 unit tests in `tests/`, covering positive, negative and numerical cases, pass on Linux (GCC 13) and on the Windows build (MinGW-w64, run under Wine). CTest runs the same executable. Malformed fixtures live in `tests/data/`.

`docs/walkthrough/` holds 17 timestamped screenshots and a log, captured by driving the **Windows** `Launch.exe` with Playwright. They show:

- STL import and the rejection of NaN and binary files;
- the FEA study, its validation and convergence;
- the synthetic point cloud with node inspection, and duplicate-id rejection;
- benchmark progress, cancellation and results;
- device connection, timeout and recovery;
- the backprop check.

---

7. Limitations

- Only ASCII STL is supported. The CSV example is synthetic. Mesh/result registration is not done.
- Hardware is simulated end to end.
- Performance figures come from a shared VM and vary by roughly ±30 %.
- `Launch.exe` is unsigned (SmartScreen prompt), and it was verified under Wine. A final check on physical Windows 11 is required before submission.
- Large point clouds (≳ 1 M nodes) depend on GPU drawing speed. That was not measured here because only a software renderer was available.